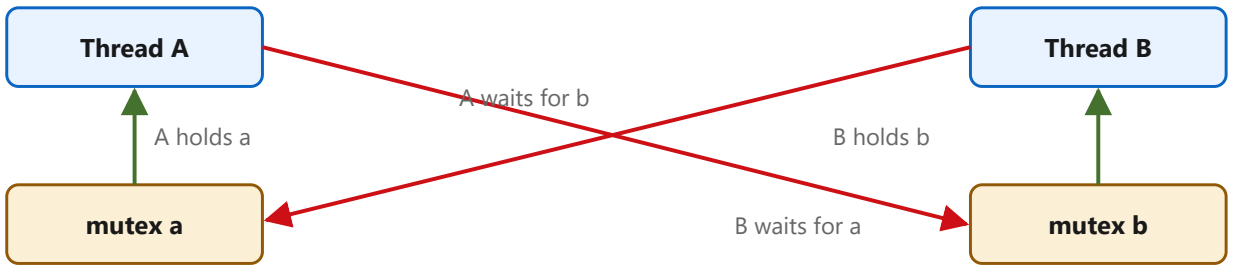
Deadlock

Why threads can wait for each other forever, the four conditions, the wait-for graph, a global lock order, `scoped_lock` and the guidelines that prevent it. Assumes Mutexes and Locking.

Deadlock

A **deadlock**: threads wait for each other forever. A holds `a` wants `b`; B holds `b` wants `a`.

STEP	THREAD	RUNS	STATE AFTERWARDS
1	A	`a.lock()`	A holds a
2	B	`b.lock()`	B holds b
3	A	`b.lock()`	A waits for b, which B holds
4	B	`a.lock()`	B waits for a, which A holds: **deadlock**, neither can ever continue



No thread can move: the arrows form a cycle.

A cycle in the wait-for graph is a deadlock.

"Run 500 random schedules": many end in deadlock. In real programs it may occur once in a million runs, only in production.

It needs all four (Coffman conditions); remove one and it cannot happen:

1. **Mutual exclusion**: one holder at a time
2. **Hold and wait**: holds one lock, waits for another
3. **No preemption**: a lock cannot be taken away
4. **Circular wait**: A waits for B, B for A

Fix: one global lock order

If **every** thread takes `a` before `b`, no cycle is possible.

```
std::mutex a, b;

// C++17: locks both with a deadlock-avoidance algorithm
void thread1() { std::scoped_lock lk(a, b); /* ... */ }
// OK even though the order differs: scoped_lock avoids the deadlock
void thread2() { std::scoped_lock lk(b, a); /* ... */ }

// pre-C++17:
// lock both atomically (deadlock-free)
std::lock(a, b);
// adopt the already-held locks so RAII still unlocks
std::lock_guard<std::mutex> la(a, std::adopt_lock);
std::lock_guard<std::mutex> lb(b, std::adopt_lock);
```

Four guidelines, one idea

Do not wait for a thread that might be waiting for you.

1. **Avoid nested locks.** If you need several, take them in one step with `std::scoped_lock`.

2. **No user-supplied code under a lock.** A callback may take another lock, or keep a pointer to your data:

```
class Wrapper {
    Data data_;
    std::mutex m_;
public:
    template <typename F>
    void process(F f) {
        std::lock_guard<std::mutex> lk(m_);
        // hands a reference to the protected data to unknown code
        f(data_);
    }
};

Data* leaked;
// the callback keeps the pointer
wrapper.process([](Data& d) { leaked = &d; });
// later: modifies the data with NO lock held
leaked->modify();
```

   Never pass out a pointer or reference to protected data.

3. **Fixed lock order.** Hand-over-hand locking on list nodes deadlocks when two threads walk in opposite directions. Fix the traversal direction.
4. **Lock hierarchy.** Each mutex has a level; a thread may only lock a *lower* level than it holds. Violations throw the first time, even if the unlucky timing never occurs. It is a small wrapper around `std::mutex` that records the current level in a `thread_local` variable and throws if you lock upward. The idea is worth knowing; in practice, `std::scoped_lock` and a documented lock order cover most programs.

NO MUTEX NEEDED Two threads that `join()` each other deadlock, as does any cycle of futures or condition-variable waits.

```
// both threads are started elsewhere, running funcA and funcB
std::thread tA, tB;

void funcA() { tB.join(); }          // A waits for B to finish ...
//  ... and B waits for A to finish: neither ever returns
void funcB() { tA.join(); }
```

KEY POINT "Prevent deadlock?" Global lock order; `std::scoped_lock`; no foreign code under a lock; short locks; no nesting; `try_lock` with back-off. Re-locking a non-recursive mutex you own is undefined behavior.

Livelock, starvation and lost wakeups

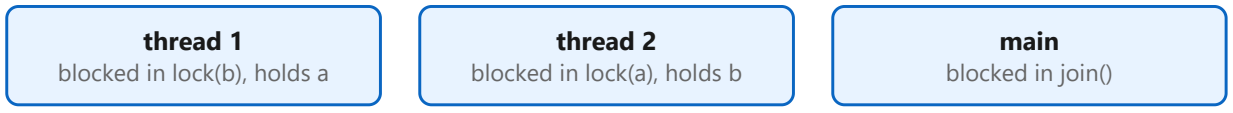
Deadlock is the loudest bug but not the only way threads fail to make progress.

PROBLEM	WHAT HAPPENS	CPU	TYPICAL CAUSE
deadlock	threads wait for each other forever	idle	two locks taken in opposite orders
livelock	threads keep reacting to each other and make no progress	busy	both back off and retry in lockstep
starvation	one thread never gets the resource	busy elsewhere	an unfair lock or a priority inversion
lost wakeup	a waiter sleeps through its signal	idle	notify before wait without a predicate

- Livelock is cured by randomizing the retry delay or by giving the threads a fixed order.
- `std::mutex::try_lock` never blocks: use it to back off instead of waiting, but then handle failure.

Finding a deadlock

A deadlock rarely announces itself: the program just stops. Finding it is a matter of asking every thread where it is.



CPU: 0%. No thread can make progress, so nothing moves and nothing crashes.

gdb: thread apply all bt shows each thread's stack; look for who waits on which mutex.

TSan with -fsanitize=thread also reports lock-order inversions before the hang happens.

A deadlock shows up as threads waiting, the CPU idle, and a cycle in who holds what.

TOOL	WHAT IT SHOWS
`gdb` : `thread apply all bt`	the stack of every thread: look for several threads inside `lock` , `wait` or `join`
ThreadSanitizer (`-fsanitize=thread`)	lock-order inversions (a before b in one place, b before a in another), even if no hang occurred
a lock hierarchy in debug builds	throws the first time a thread locks against the order
logging with thread ids	which thread took which lock, in what order

REPRODUCE IT Add a short `sleep_for` between the two lock calls while debugging: it widens the window so the deadlock happens every time instead of once a month.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Deadlock	C++ Concurrency in Action	3.2.4 to 3.2.5 Deadlock: the problem, the solution and further guidelines	74 to 82